



The Transport Layer — Multiplexing, Ports, UDP, and the TCP Connection

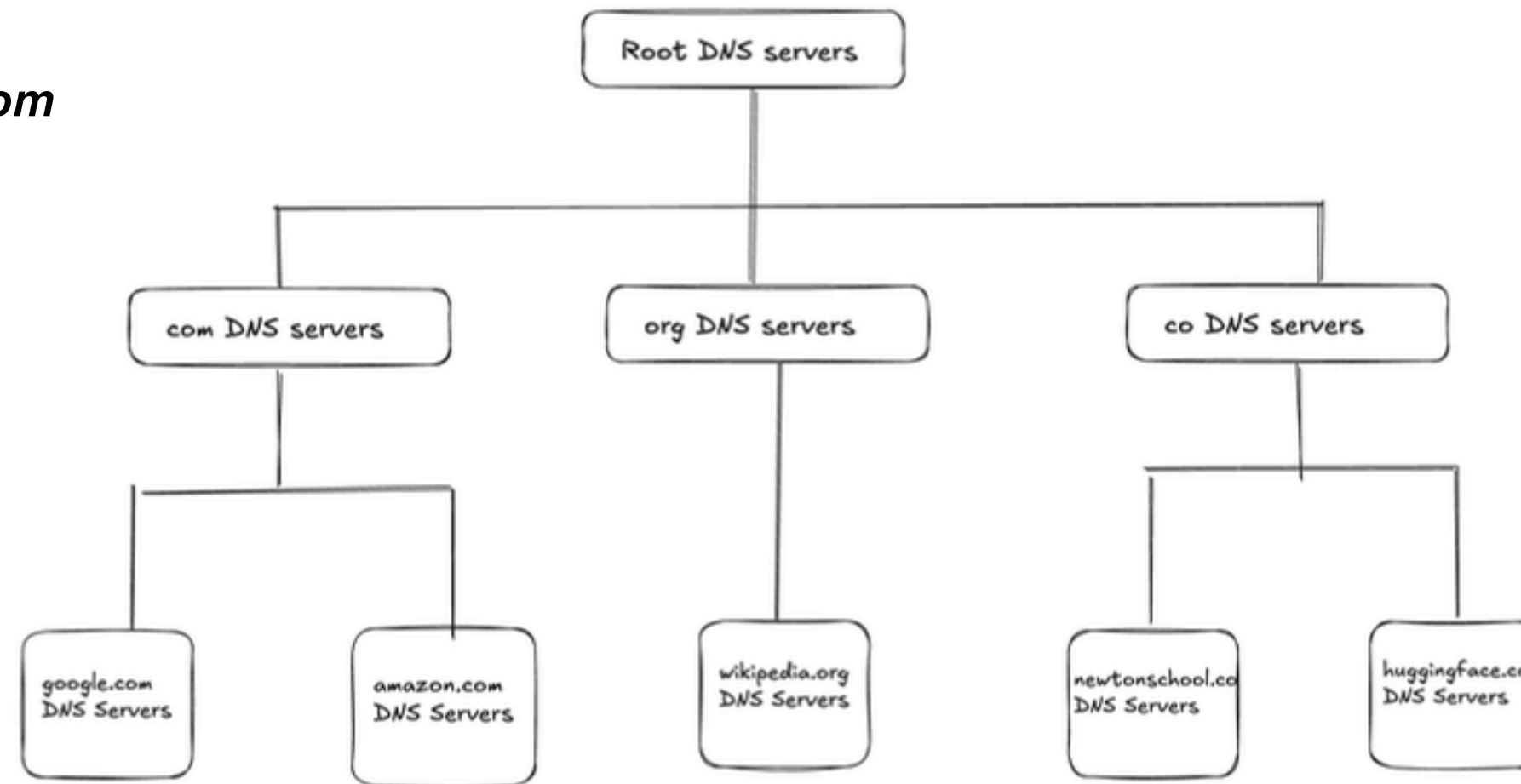
Computer Networks

Join the lecture online on your dashboard

Recap

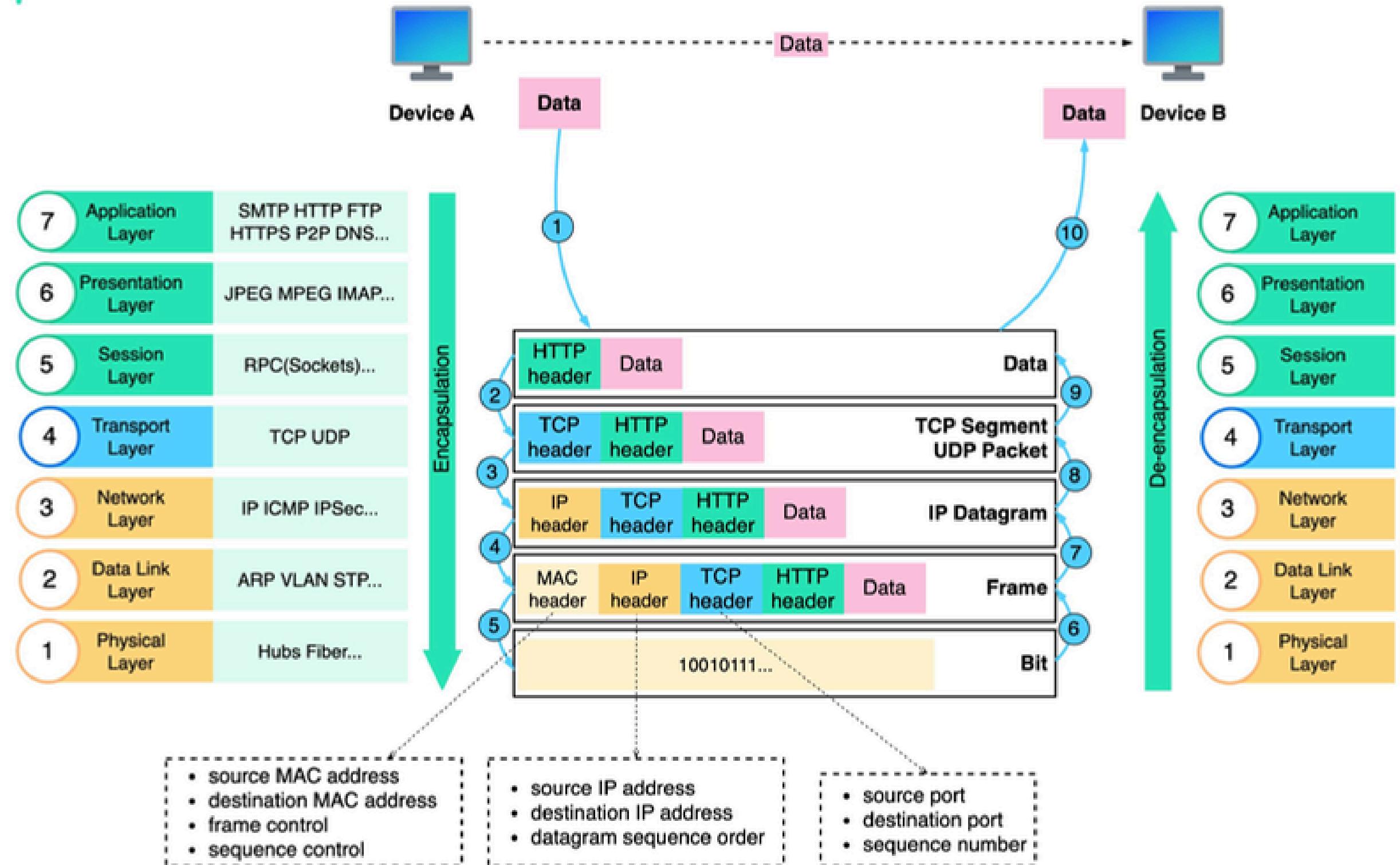
How DNS Lookup Works

Example: www.amazon.com



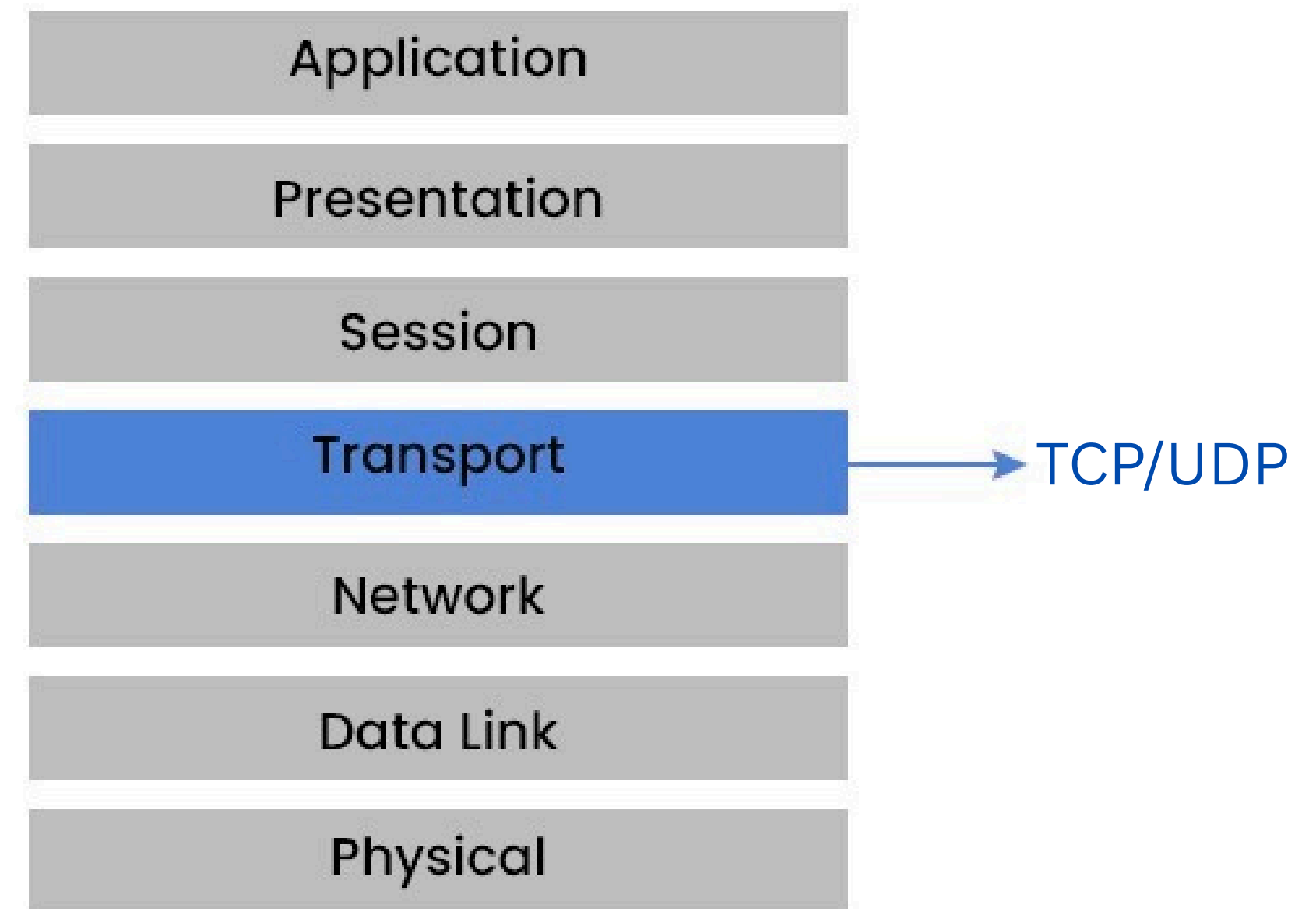
1. Client contacts Root DNS server → returns IP addresses of TLD servers (.com)
2. Client contacts TLD server (.com) → returns IP of authoritative server (amazon.com)
3. Client contacts authoritative server (amazon.com) → returns IP of `www.amazon.com`

Recap



Layer 4

- Implemented in sending and receiving devices
- Network routers only process network-layer information
- Routers ignore transport-layer segment fields completely



Transport layer converts application messages into segments

Ports and Sockets

Ports and Sockets

The Apartment Building Analogy

Sending a letter to a friend living in a large apartment building requires:

- 🏢 Building's street address (1 Infinite Loop)
- 🚪 Friend's apartment number (Apartment #443)

Without both pieces, your letter gets lost!

How This Maps to Networking

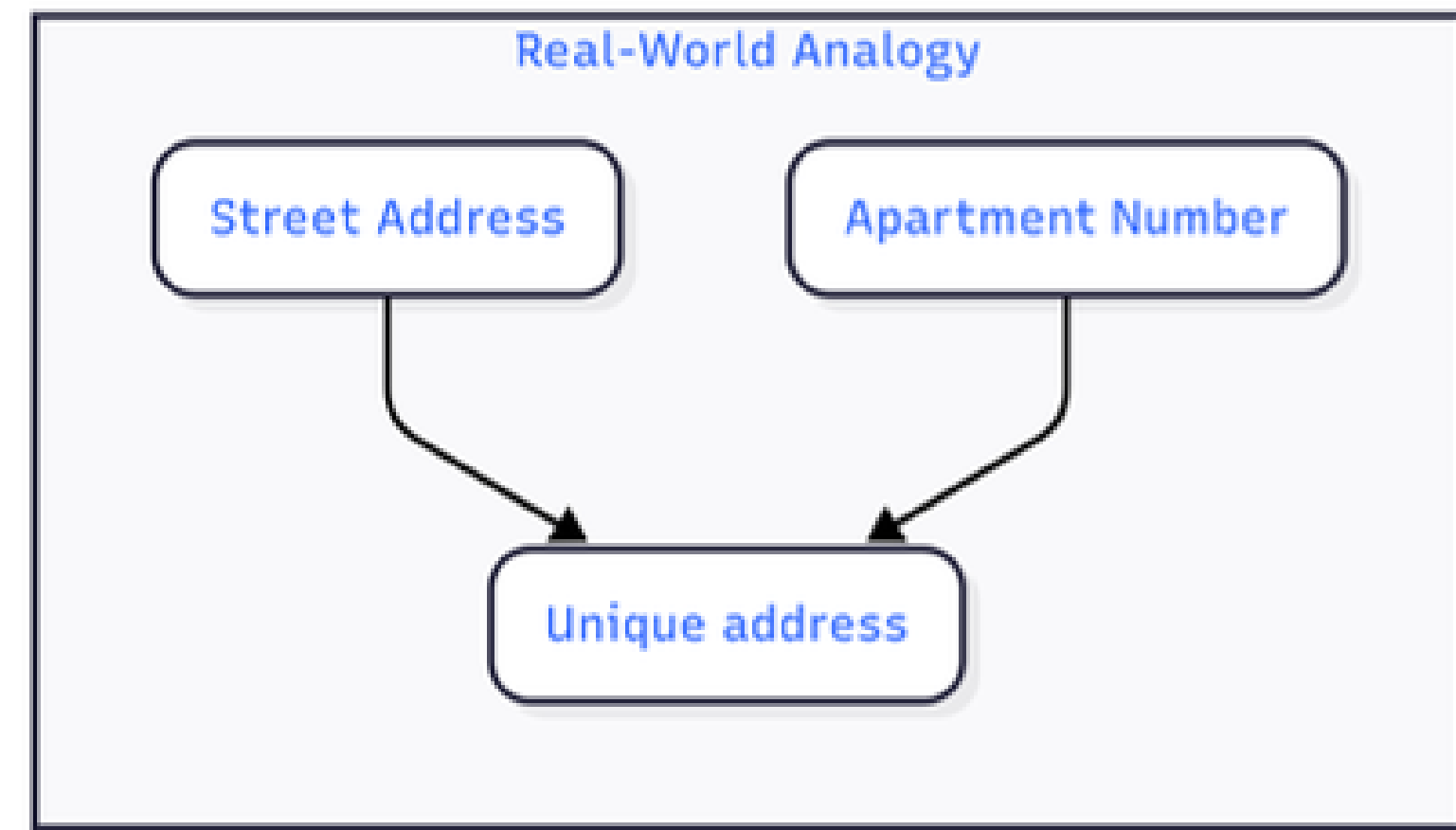
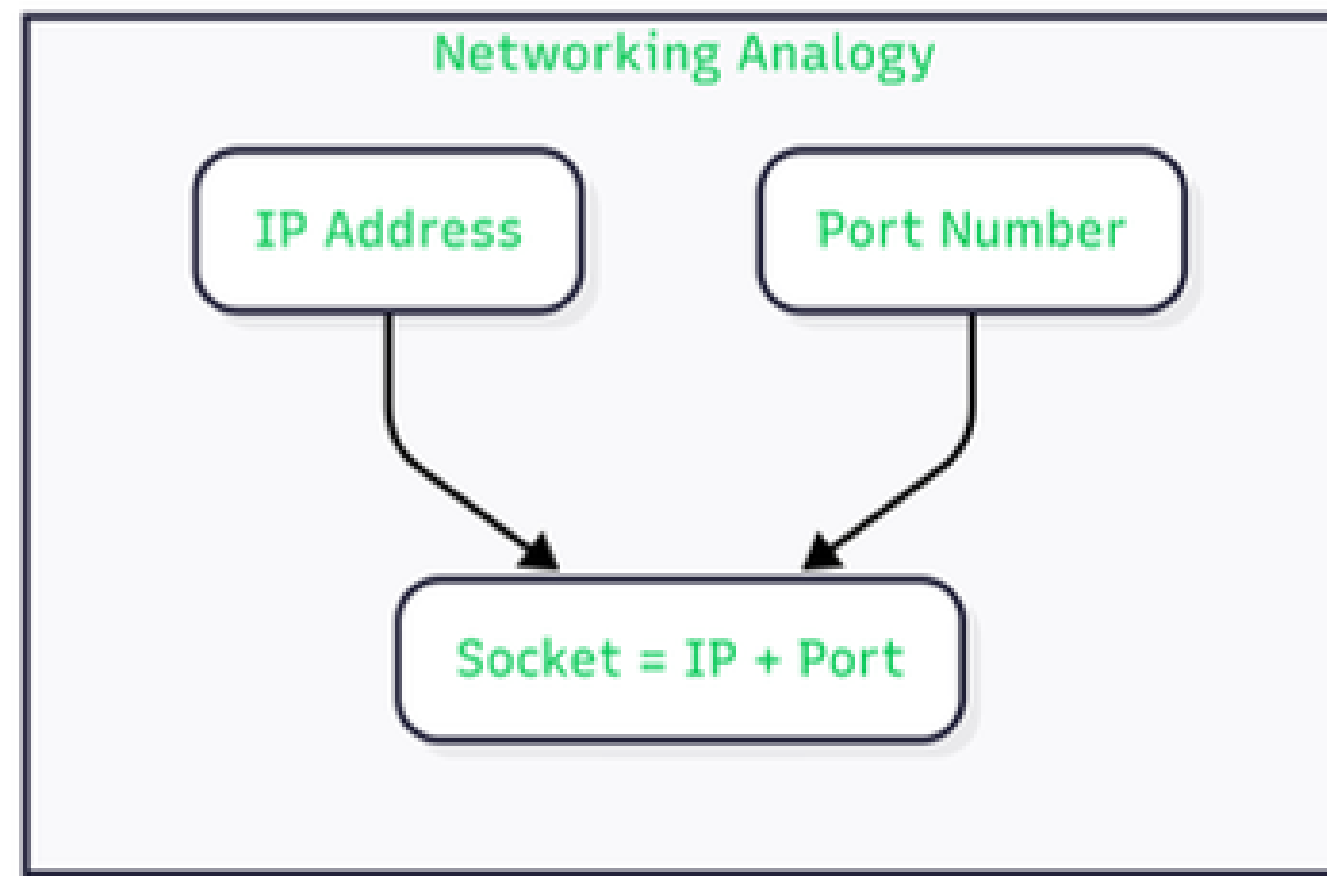
In networking, it works exactly the same way:

- 🏢 IP Address = Computer's "street address" (172.217.167.78)
- 🚪 Port Number = Application's "apartment number" (Port 443)

IP Address + Port Number = Socket (unique endpoint)

Example: Your browser connecting to Google
Your Laptop IP:Random Port ↔ Google IP:443

Ports and Sockets

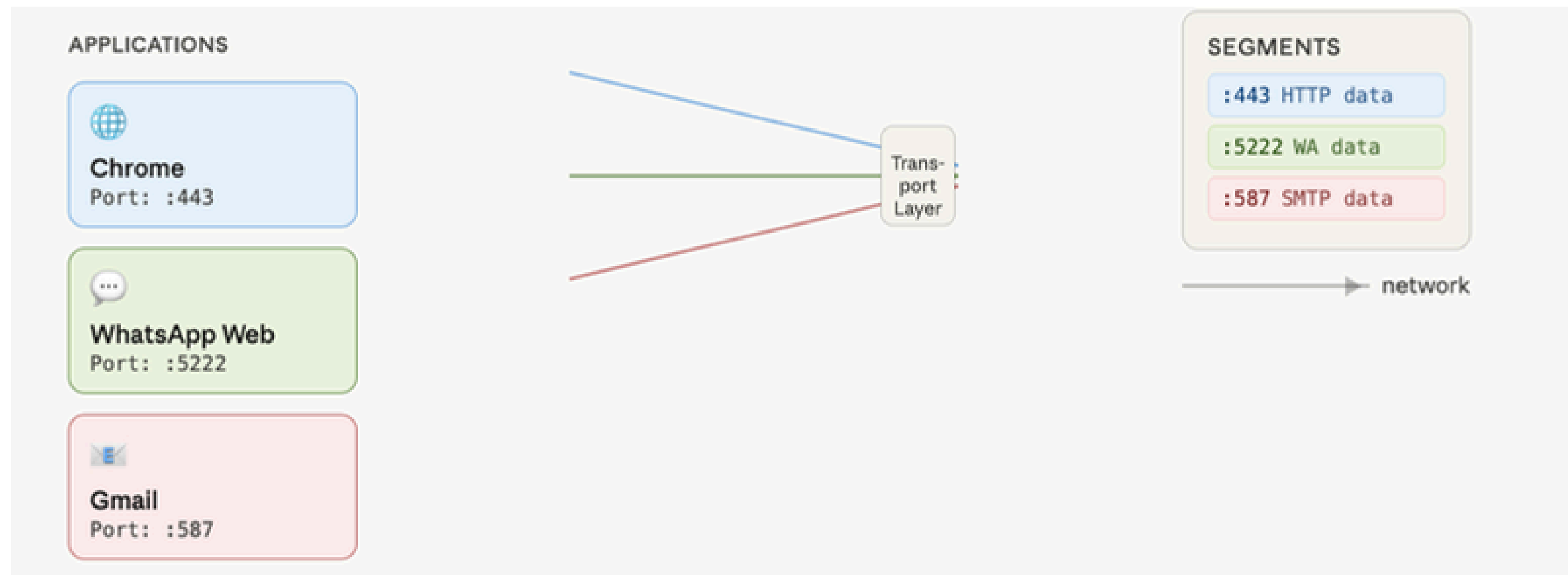


When you combine an IP address and a Port number, you create a unique endpoint for a conversation. This specific "doorway" on a computer, is called a Socket.

netstat -tnlp tcp

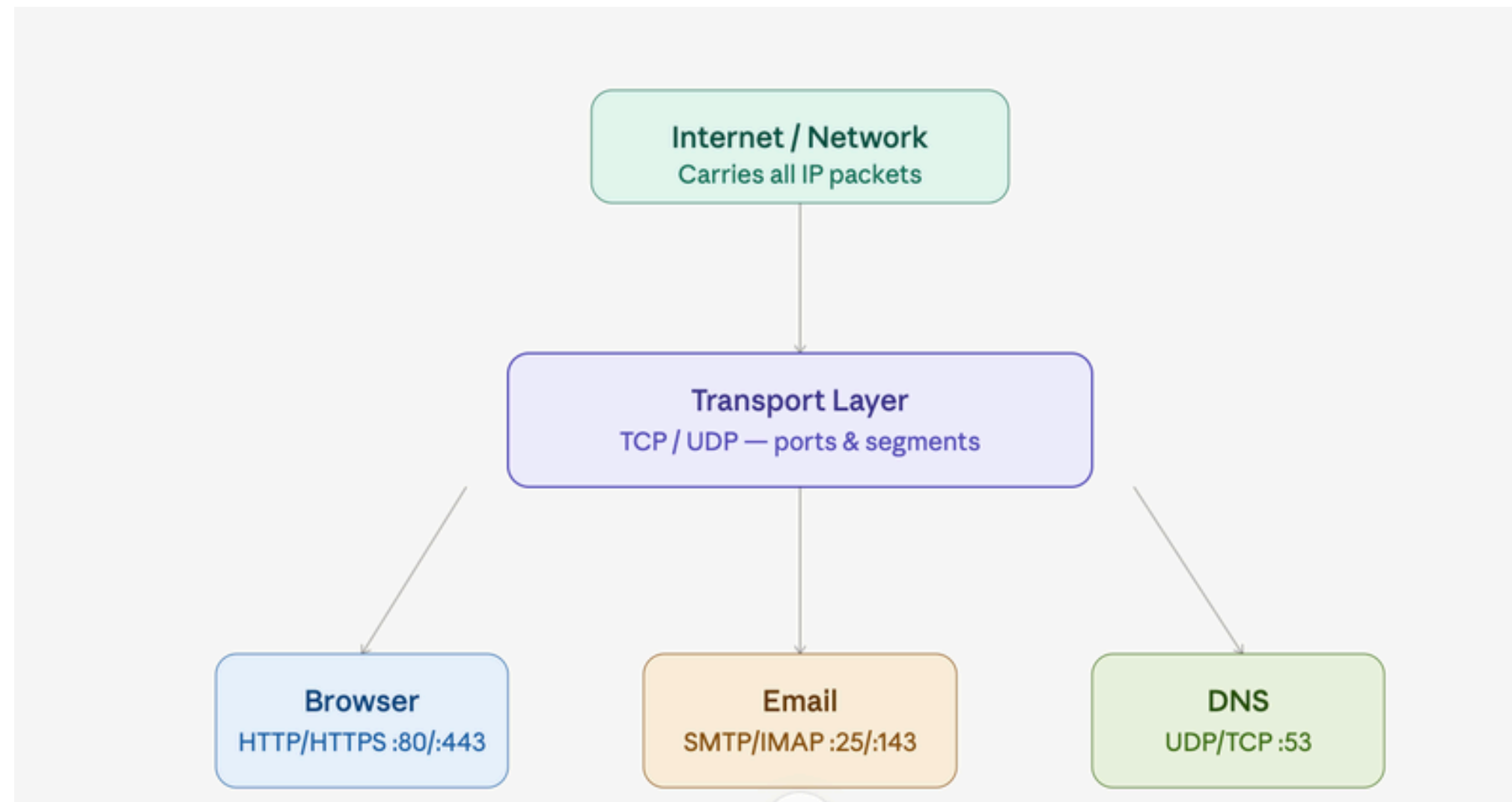
Multiplexing (Sender Side)

Multiplexing is the process of collecting data from multiple application processes and sending it through the network using transport layer protocols.



Demultiplexing (Receiver Side)

Demultiplexing is the process of delivering received data to the correct application process using port numbers.



Transport Layer Protocols

UDP Protocol

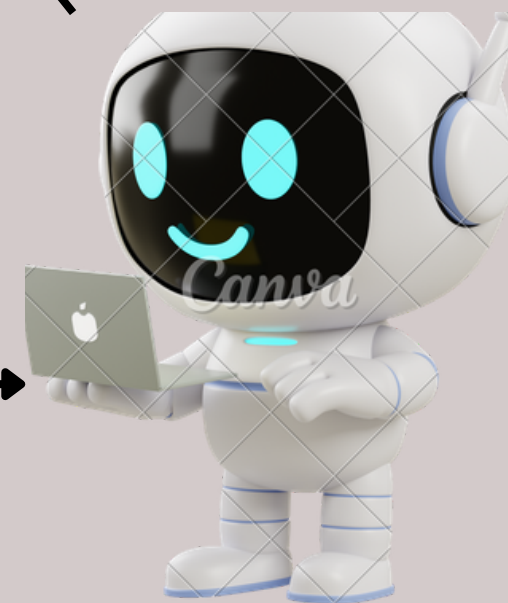
UDP

Are you
getting all of
this?

Who cares
just sent it
faster!

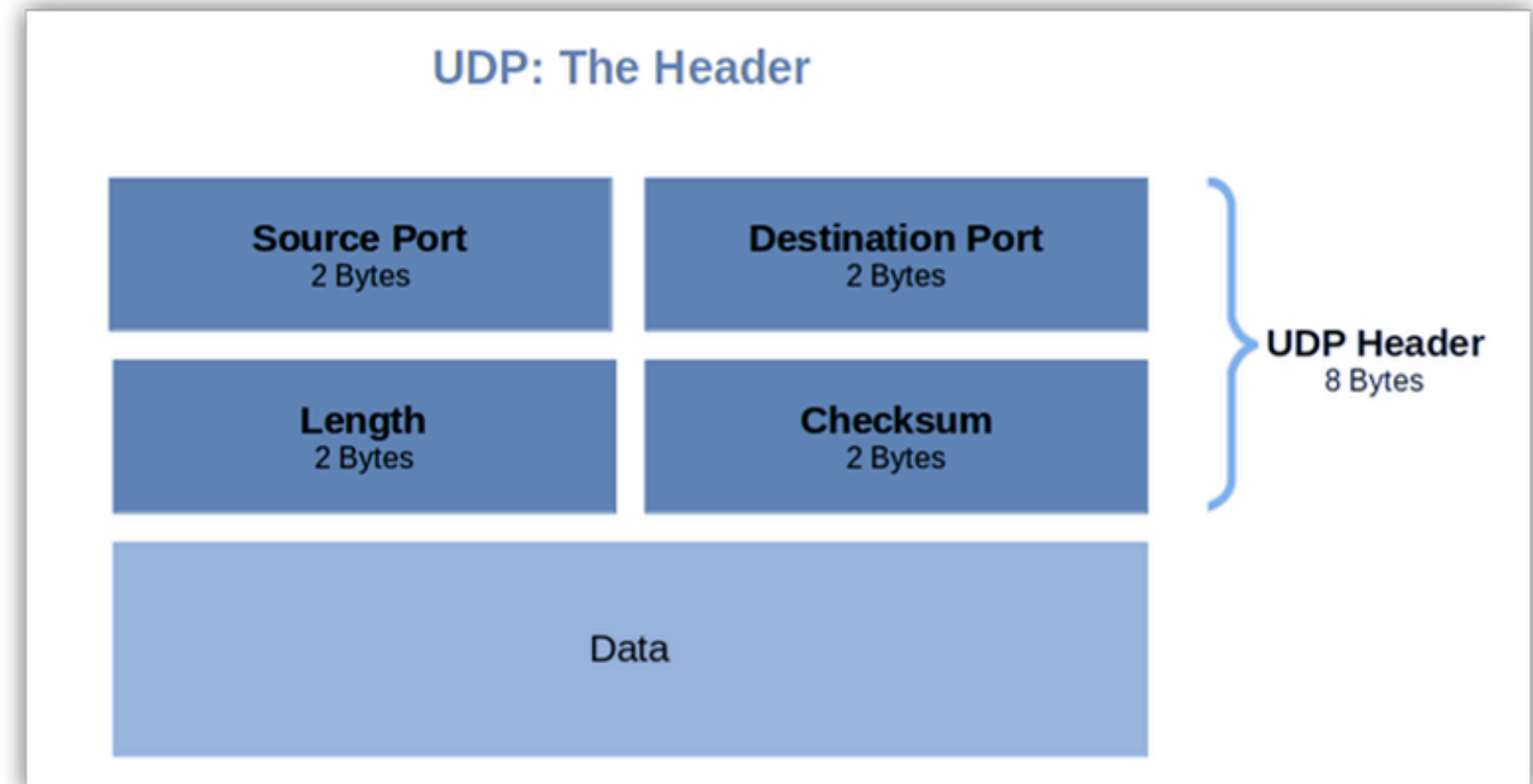
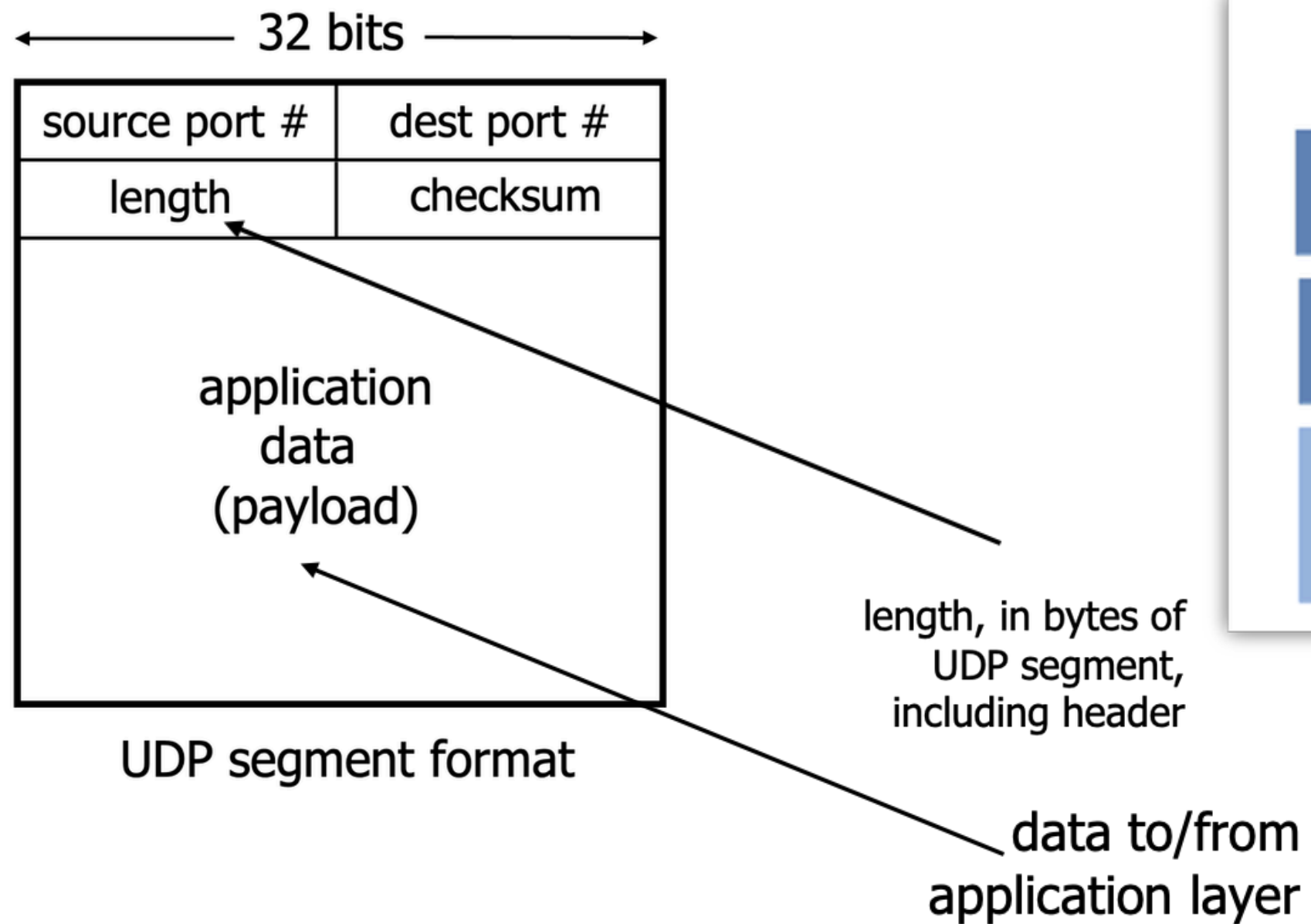


Sender

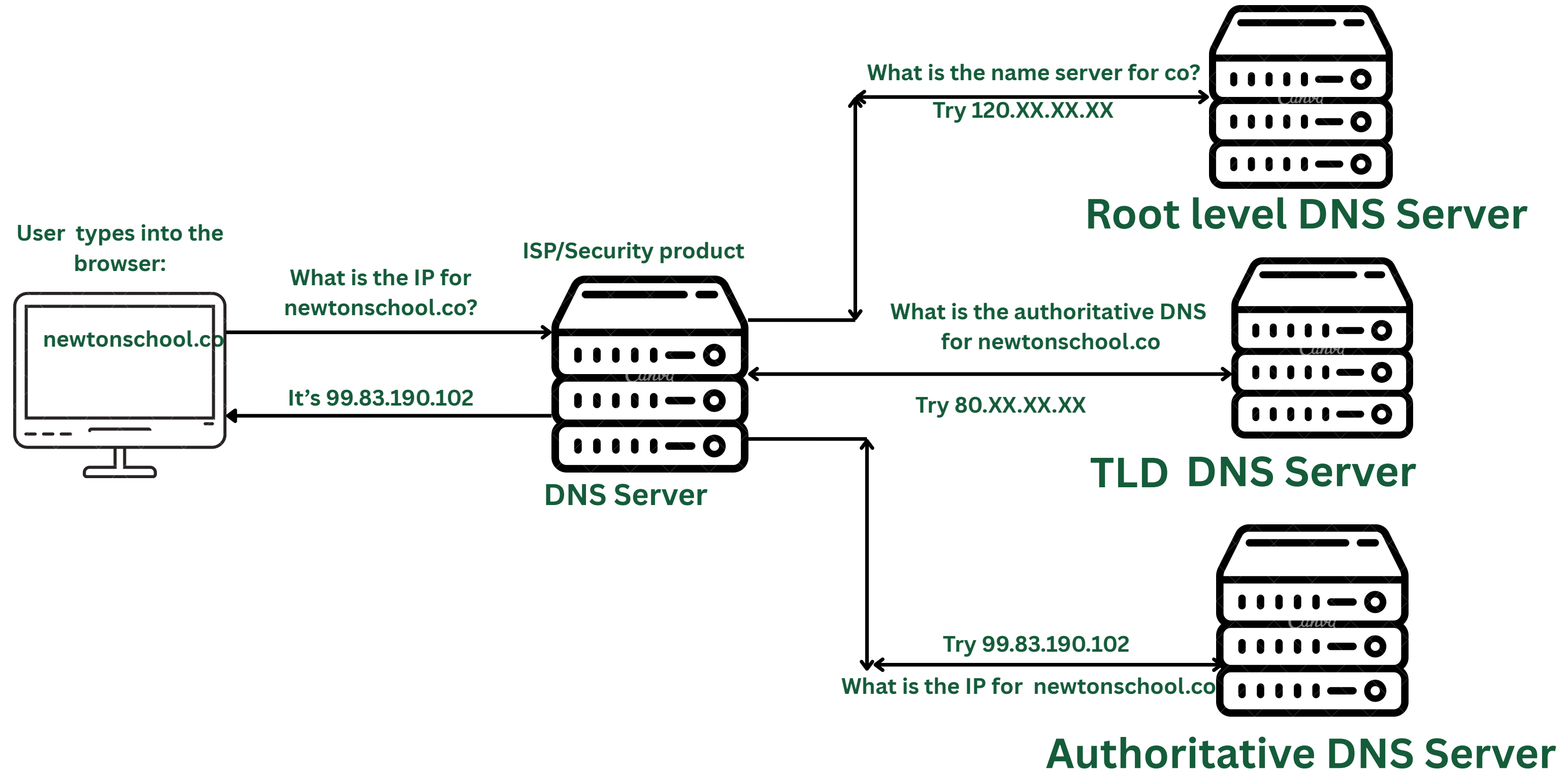


Receiver

UDP Segment



Domain Name System(DNS)



Video streaming

- Live streaming involves watching events in real-time, such as cricket, news, or gaming, Unlike buffered YouTube videos, live streams prioritize speed.
- [Why use udp?](#) using UDP to send video packets quickly without delays.
- If some frames are lost, they are skipped to avoid lag. The goal is to maintain a smooth and instant video flow.

TCP & The 3-Way Handshake

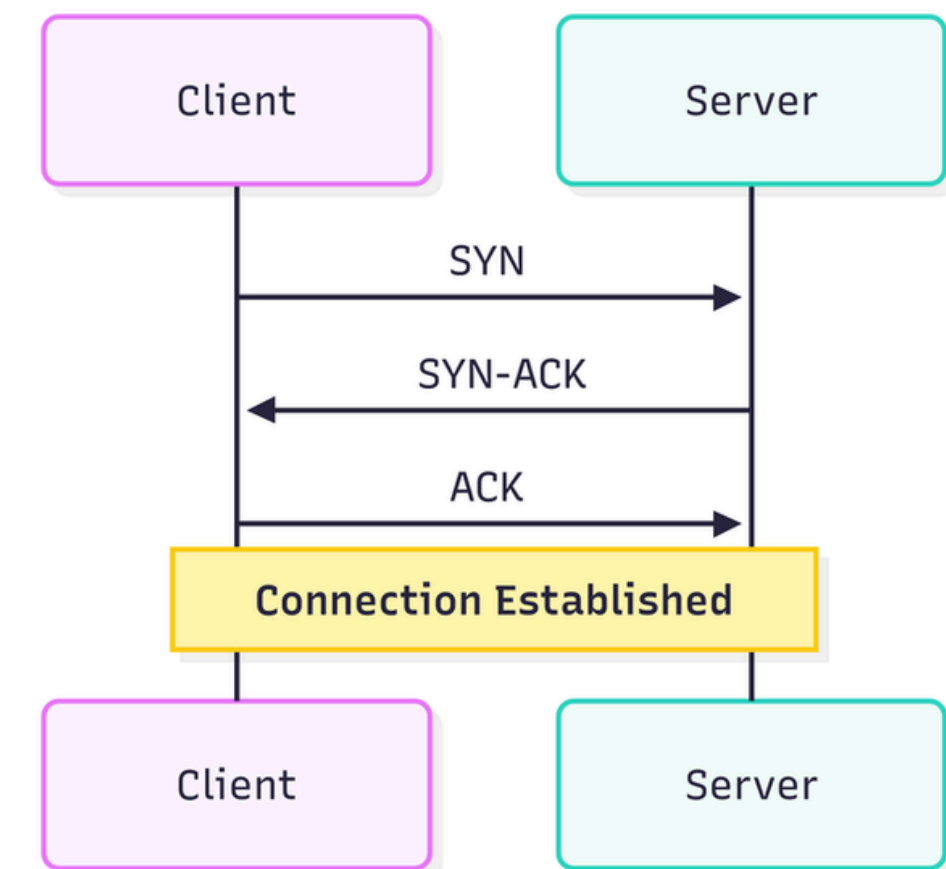
TCP 3-Way Handshake Ritual

Let's tell this story from the perspective of your browser (client) connecting to google server...

Step 1: The Ring 📞 (SYN)

- Your browser calls the Google server by sending a special, tiny message called a SYN packet. SYN stands for "**synchronize**." It's your browser essentially saying:

"Hey, are you there? I'd like to start a conversation. To keep our chat organized, I'll start numbering my sentences at number 100."

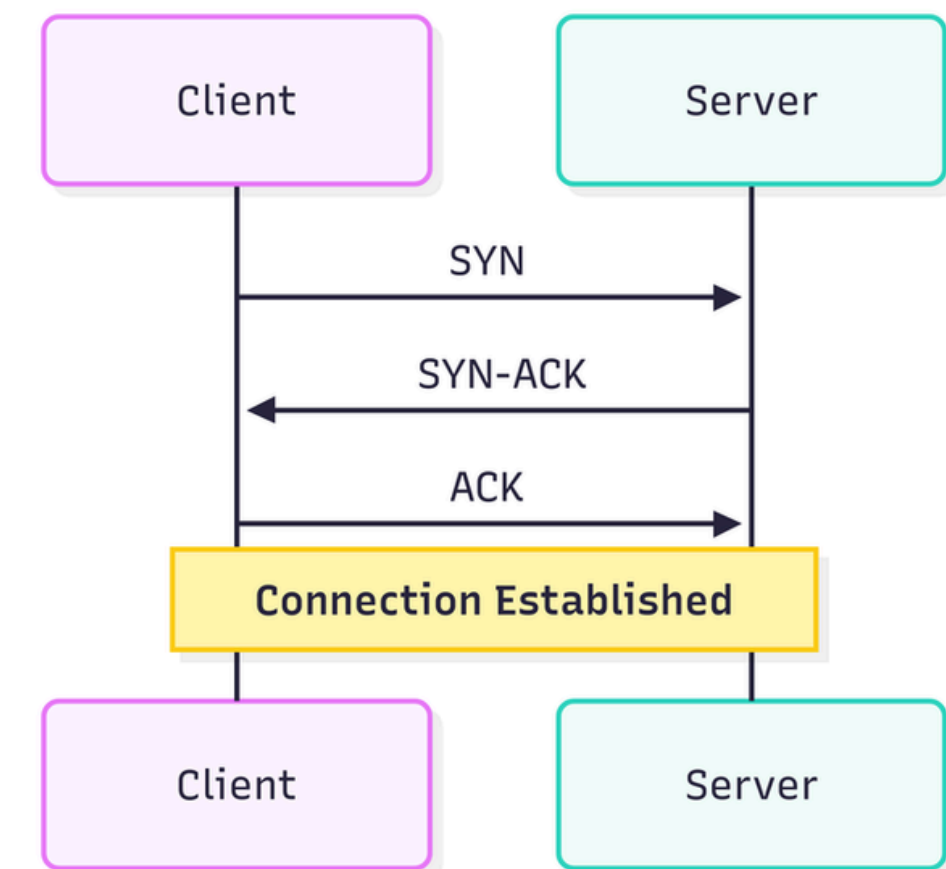


TCP 3-Way Handshake Ritual

Step 2: The "Hello?" 🖐️ (SYN-ACK)

- The Google server, which is always listening for calls on Port 443, gets the SYN packet. It responds with its own special message, a SYN-ACK packet. This message does two things at once:

*"Hi! Yes, I'm here! I **ACKnowledge** your request to start at sentence number 100. And to keep my side of the chat organized, I'll **SYNchronize** and start my sentences at number 350. Are you ready?"*



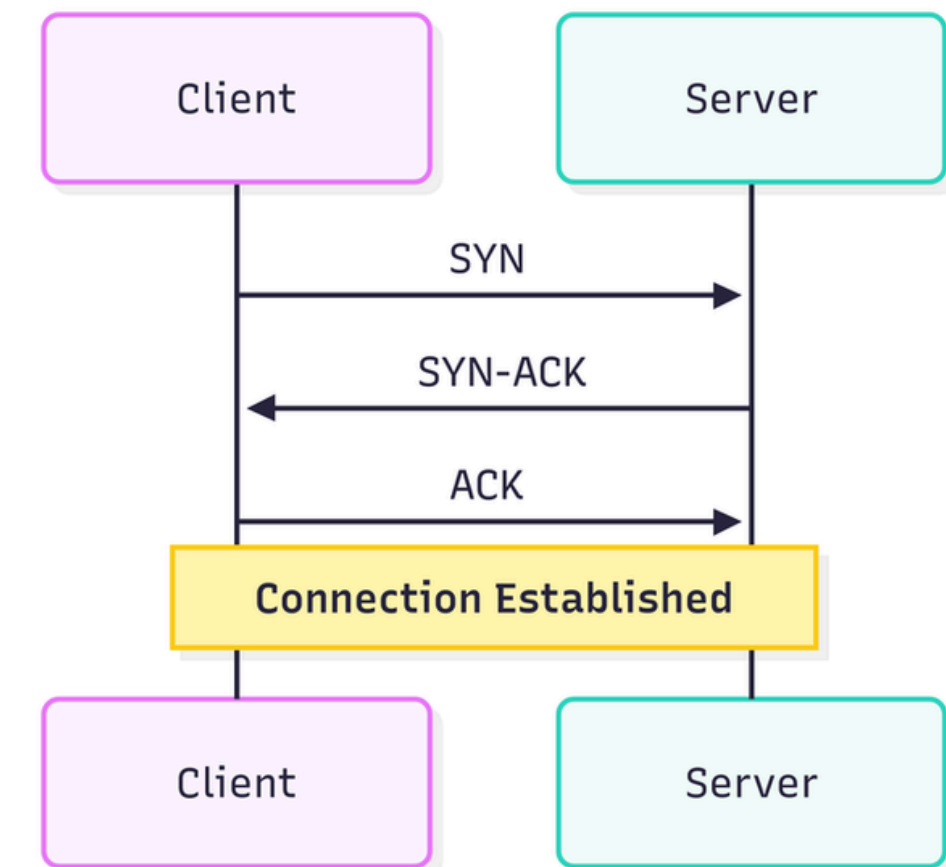
TCP 3-Way Handshake Ritual

Step 3: The "I'm Here!" 👍 (ACK)

- Your browser gets the server's response. The session is almost established! It just has to do one last thing: confirm it heard the server. It sends a final ACK (acknowledgment) packet.

"Got it! I ACKnowledge that you're starting at 350. Let's talk!"

- And boom! The connection is ESTABLISHED.



TCP Connection Teardown

TCP has a polite "goodbye" process. It's usually a 4-Way Handshake.

Step 1: The "I have to go." (FIN)

The client sends a FIN packet (for "finish")

Step 2: The "Okay." (ACK)

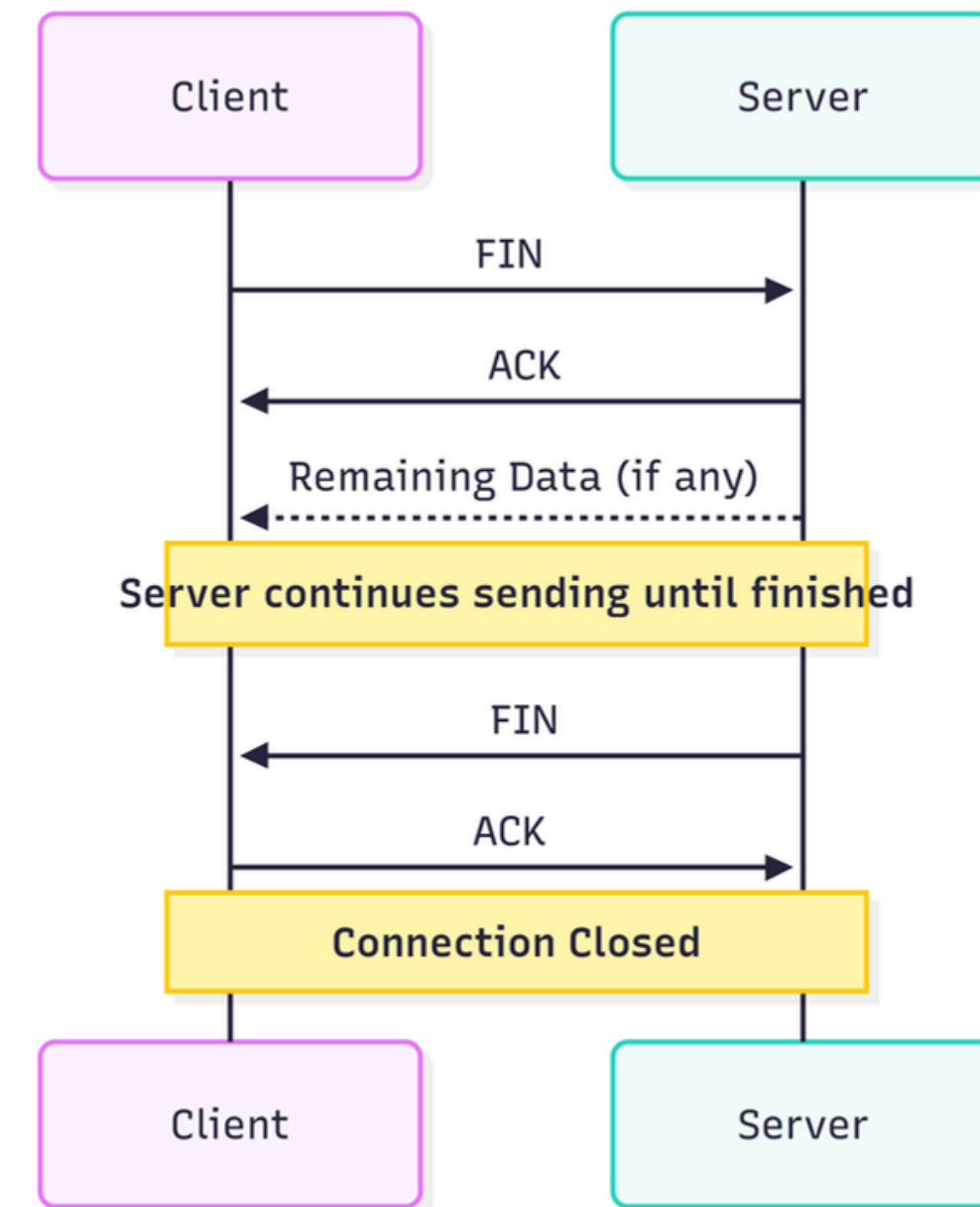
The server receives this and acknowledges it.

Step 3: The "I'm done too." (FIN)

Once the server has finished sending any remaining data, it sends its own FIN packet.

Step 4: The Final "Bye." (ACK)

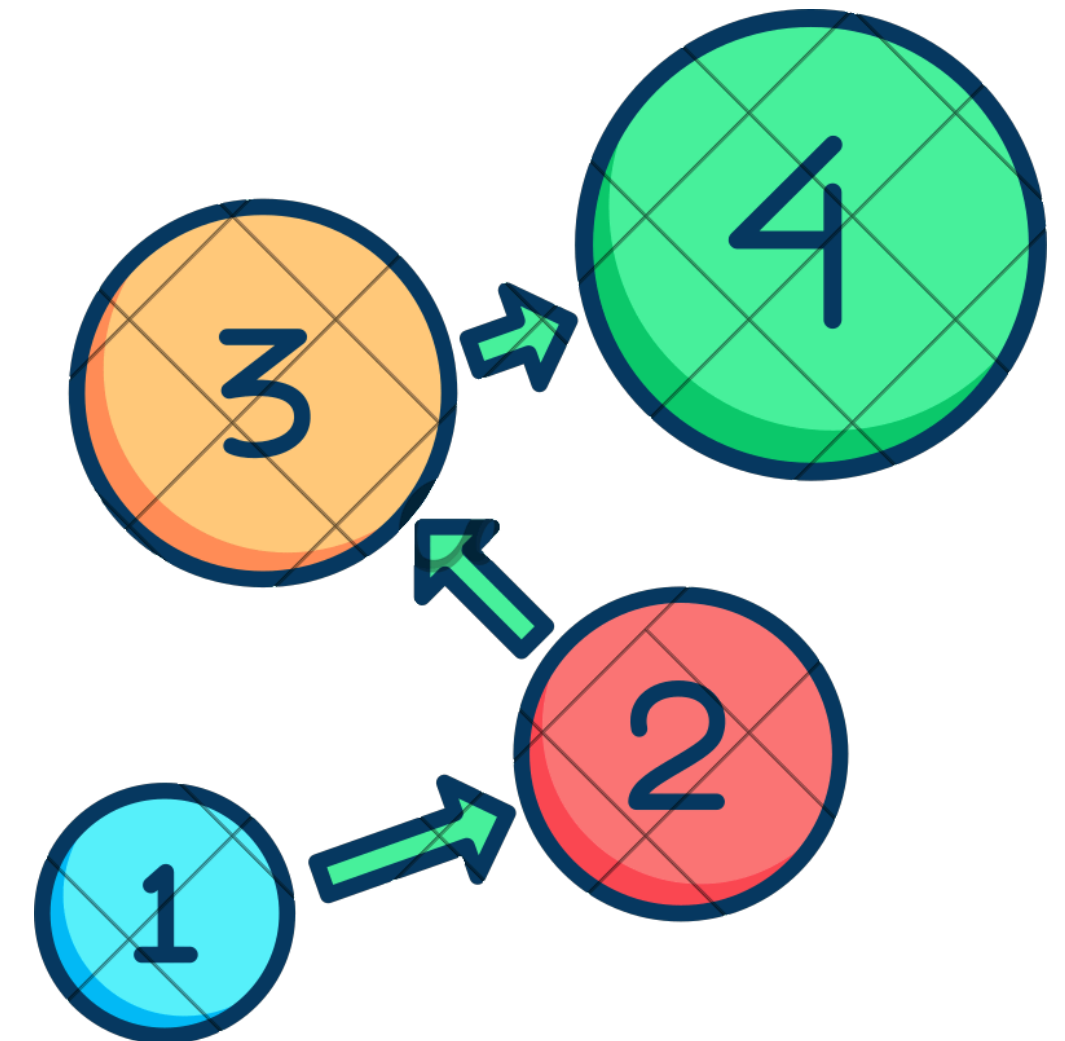
Your client gets the server's FIN and sends one last ACK.



Sequence Numbers

Ordered Delivery

- Messages can take different routes and arrive out of order. Imagine trying to read a story that came in like this: "the mat." "The cat" "sat on". It's confusing!
- TCP solves this by using the sequence numbers we established during the handshake.
- Every single piece of data (segment) is given a number. If the receiver gets segment #3 before segment #2, it simply waits for #2 to arrive before putting the story back together in the correct order for the application.



Acknowledgments & Retransmission

How do you know the other person heard you on a phone call?

You listen for them to say "hmm," or "achcha."

- When the server receives a chunk of data (say, segments 101-200), it sends back an ACK message that says, "I've received everything up to number 201."
- If your browser doesn't get an ACK back within a certain amount of time, it assumes the data was lost. **What does it do?**



It simply retransmits the lost data. It sends it again, saying, "Did you get that part?"

TCP Segment: Preview

Legend:

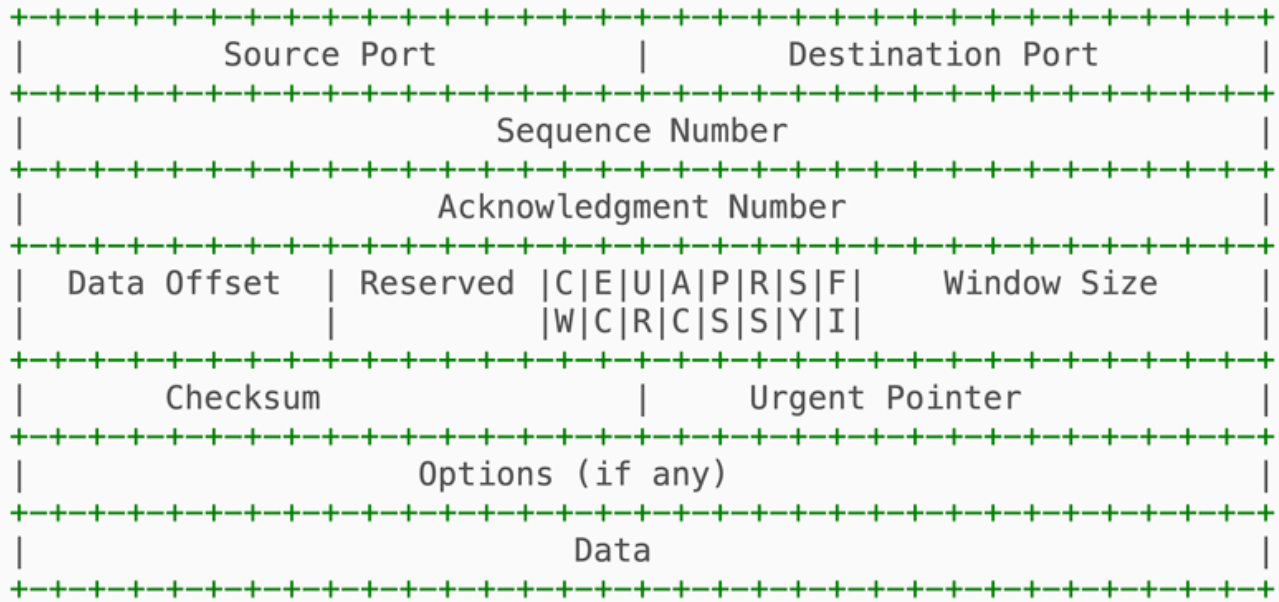
Source Port: Specifies the port number of the sending application.

Destination Port: Indicates the port number of the receiving application.

Sequence Number: Provides a unique identifier for each TCP segment to ensure ordered delivery.

Acknowledgment Number: Acknowledges the receipt of data and indicates the next expected sequence number.

Data Offset: Specifies the length of the TCP header in 32 bit words.



TCP Segment: Preview

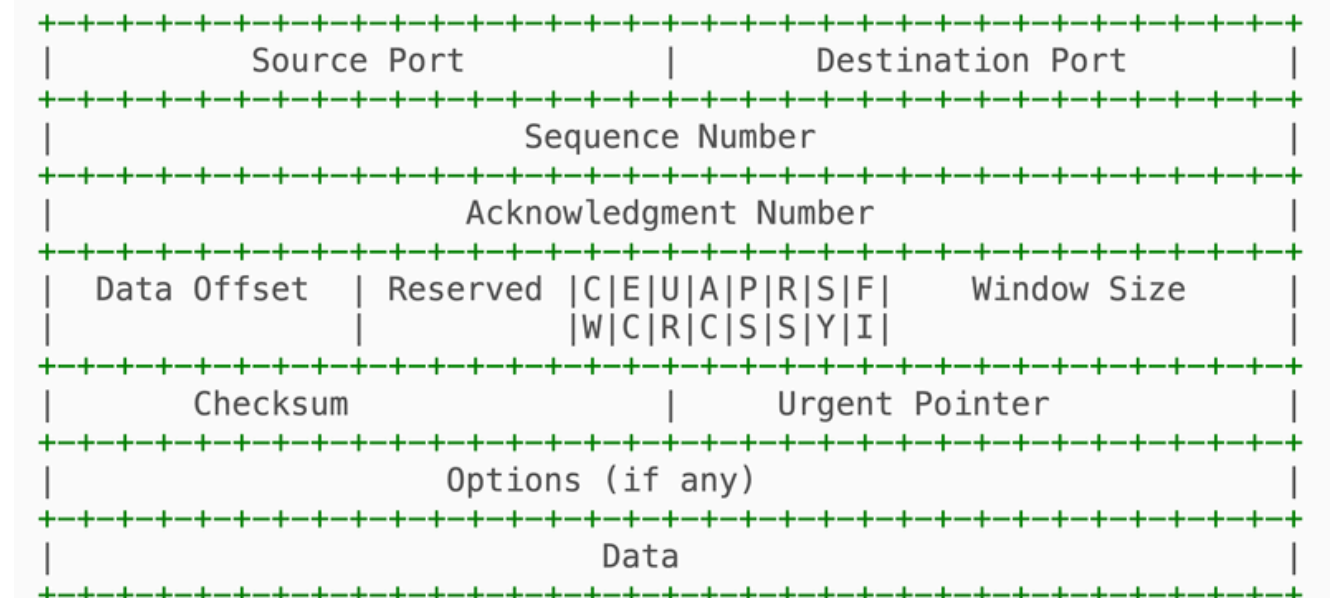
Legend:

Control Flags: These flags include various control and status bits such as SYN, ACK, FIN, etc.

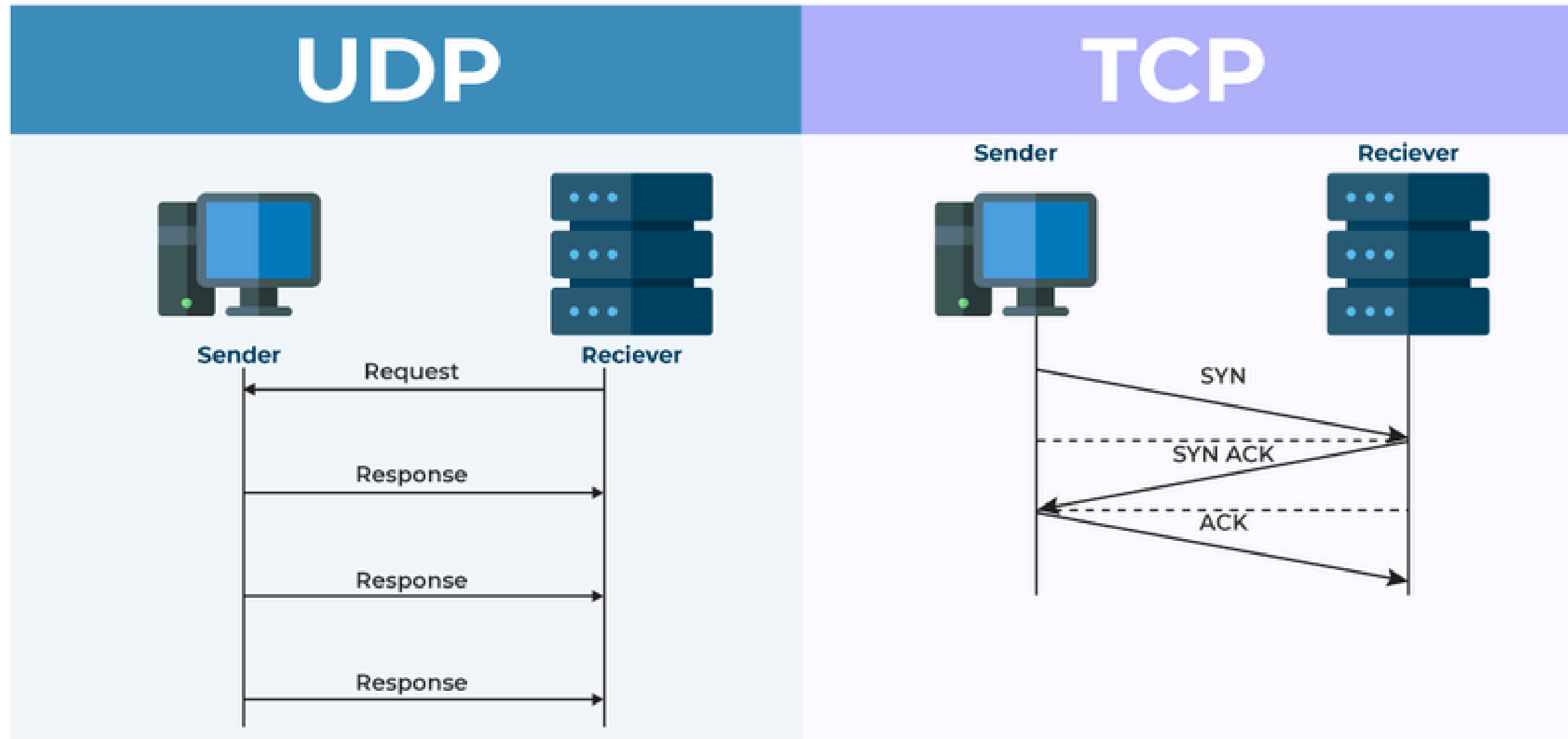
Window Size: Indicates the receiver's buffer size for flow control.

Checksum: Helps ensure data integrity during transmission.

Urgent Pointer: If the URG flag is set, then this 16-bit field is an offset from the sequence number indicating the last urgent data byte.



Speed vs Reliability – The UDP Trade-Off



Please fill the feedback form.

Thank You